

Deploys & Reminders

Detailed specification of two new dashboard modules — every feature, every form, every state, every helper, every seed entry, every helper function, every interaction.

VERSION	BRANCH	STATUS	PAGES	MODULES	FILES	LOC
0.9.0	develop	UI complete	~20	2	14 added · 8 modified	~1,560

Contents

00	Executive summary
01	Module — Deploys
1.1	Purpose & user roles
1.2	Information architecture
1.3	Page header
1.4	Statistics row
1.5	Filter bar
1.6	Latest hero card
1.7	Timeline list
1.8	Changelog component
1.9	Upload dialog
1.10	Status & environment models
1.11	File-type iconography
1.12	Helper functions
1.13	Download flow
1.14	Empty states
1.15	Data model
1.16	Store
1.17	Seed data inventory
02	Module — Reminders
2.1	Purpose & user roles
2.2	Information architecture
2.3	Page header
2.4	Statistics row
2.5	Status tabs
2.6	Filter bar
2.7	Reminder card anatomy
2.8	Urgency model
2.9	Add / Edit dialog

	2.10 Auto-expiration logic
	2.11 Interval format display
	2.12 Card actions
	2.13 Analytics & counts
	2.14 Empty states
	2.15 Data model
	2.16 Store
	2.17 Seed data inventory
03	Shared architecture
04	Backend integration roadmap
05	Appendices · file index, commits, glossary

00 · Executive summary

Two pages were added to fill operational gaps the rest of the platform did not address:

- **Deploys** — backend, frontend, and mobile engineers need a single place to drop a build artifact, write a short changelog, and surface it to managers. The page records every upload with file metadata, environment (production / staging / preview), status (ready / deprecated / rolled-back), and an expandable bullet changelog. The most recent upload is rendered as a hero card; everything older sits on a vertical timeline.
- **Reminders** — subscriptions, SSL certificates, cloud reservations, domain registrations, anything with an expiry date the team needs to track. The page lets managers create a reminder with multiple "notify me before" intervals and renders each as an urgency-driven countdown card whose color shifts from green to red as the expiry approaches.

Both modules ship as *UI-only*: they read from in-memory seed data via Zustand stores and persist nothing between page reloads. The backend integration plan is included in section 04.

01 Deploys

Build-artifact distribution with a changelog timeline. Engineers upload; managers download.

1.1 Purpose & user roles

Centralize delivery of build artifacts so the team has a single canonical history of what was deployed, when, and by whom. Every upload triggers (eventually, when backend ships) a manager notification.

Primary user	Engineers uploading builds (any role: backend, frontend, mobile, ML).
Secondary user	Managers receiving notifications and downloading the latest build.
Read access	All authenticated users.
Write access	Gated by <code>p.tasks.create()</code> as a stand-in until <code>p.deploys.create()</code> is added to the permission policy.
Page path	<code>/deploys</code>
Sidebar group	Secondary (below divider), with the Rocket icon.

1.2 Information architecture

From top to bottom on `/deploys`:

1. **Page header** — title, description, primary action button.
2. **Statistics row** — four count tiles spanning the full width.
3. **Filter bar** — search input, three filter selects, clear button, live count.
4. **Latest hero card** — only when the filtered list has at least one item; the newest entry rendered prominently.
5. **Timeline list** — every other filtered entry, connected by a vertical rail.
6. **Upload dialog** (modal) — opens on "New Deploy" click.

1.3 Page header

Standard `<Header />` shared atom. The only deploys-specific elements:

- **Title:** `"Deploys"`
- **Description:** *"Build artifacts and changelog history. Managers are notified on each upload."*

- **Primary action:** Right-aligned button with `Plus` icon and label `"New Deploy"`.
Renders only if `p.tasks.create()` returns true. Opens the upload dialog.

1.4 Statistics row

Four tiles in a responsive grid (`grid-cols-2 sm:grid-cols-4 gap-3 mb-6`). Each is a `<Card>` with centered content. Layout per tile: numeric value on top (2xl, bold, tabular-nums, color-coded), tiny uppercase label below.

TILE	SOURCE FORMULA	COLOR
Total Deploys	<code>deploys.length</code>	text-text-dark
Ready	<code>deploys.filter(d => d.status === DeployStatus.Ready).length</code>	text-success
This Week	<code>deploys.filter(d => Date.now() - new Date(d.uploaded_at).getTime() < 7 × 86400000).length</code>	text-primary
Total Size	<code>deploys.reduce((s, d) => s + d.file_size, 0)</code> formatted by <code>formatSize</code>	text-text-dark

All four values are recomputed inside a single `useMemo` keyed on `[deploys]` so the component doesn't reduce four times per render.

1.5 Filter bar

A `<Card>` with `p-4` padding. Inside: a flex-wrap row of inputs and a live count.

SEARCH INPUT

Left-most element. `min-w-[200px] max-w-sm flex-1` so it absorbs slack horizontal space. `Search` icon overlaid with `insetInlineStart: 12` (RTL-safe), input padded `paddingInlineStart: 40`. Bound to `search` state. Matches against `title`, `file_name`, and `version` with a case-insensitive substring test on the trimmed query.

ENVIRONMENT FILTER

Standard `<Select>`, fixed width `w-[150px]`. Options: *All Environments* + the three enum values. Bound to `envFilter`; when not `"all"`, filters by `d.environment === envFilter`.

STATUS FILTER

Same pattern, `w-[140px]`. Options: *All Statuses, Ready, Deprecated, Rolled Back*.

UPLOADER FILTER

Dynamic options — extracted from currently loaded deploys via:

```
const uploaders = useMemo(() => {  
  const map = new Map();  
  deploys.forEach(d => map.set(d.uploaded_by.id, d.uploaded_by));  
  return Array.from(map.values());  
}, [deploys]);
```

CLEAR BUTTON

Renders only when at least one filter is active. Plain text button with an `X` icon. Resets all four filters to defaults (`""` / `"all"`).

COUNT INDICATOR

Right-aligned (`ms-auto`). Format: `{filtered.length} / {deploys.length}` in tabular numerals at `text-xs text-text-muted` .

1.6 Latest hero card

The first item of the currently-filtered list is rendered with a more prominent visual treatment than the rest of the timeline below it. If the filtered list is empty, no hero shows.

CONTAINER

- `<Card>` with `relative overflow-hidden border-primary/40`
- Background: `bg-gradient-to-br from-primary/[0.06] via-card to-card`
- Left accent rail: absolutely positioned `w-1 inset-y-0 start-0 bg-primary`
- Content padding: `p-5 ps-6` (extra start-padding to clear the rail)

FILE TYPE ICON BLOCK

56×56 rounded-lg box with `bg-primary text-primary-foreground` and `shadow-md`. Inside: 28×28 file-type icon (mapped by `iconForFile(file_name)`; see §1.11).

HEADER ROW OF BADGES

Above the title, a horizontal flex of three labels:

- `LATEST` — uppercase, tracking-wider, primary fill
- Status badge from `statusBadge(d.status)` — Ready / Deprecated / Rolled Back (see §1.10)
- Environment indicator — a 1.5×1.5 colored dot followed by uppercase 0.12em-tracked environment name, all colored by the environment's tone class (`text-success` / `text-warning` / `text-primary`)

TITLE + VERSION

`h2.text-xl font-bold text-text-dark` with the deploy title, followed by the version in `font-mono text-base text-primary tabular-nums` (only rendered if the deploy has a version). Both share the same baseline via `flex items-baseline gap-2 flex-wrap`.

FILE META LINE

One small muted line: `{file_name}` in `font-mono`, then a separator, then `{formatSize(file_size)}`. Truncated with `truncate`.

RIGHT-SIDE ACTIONS

Two controls grouped in a flex with `shrink-0`:

- **Download** — primary `<Button>` with `Download` icon. Calls `handleDownload(d)`.
- **Trash icon** — borderless icon button. Only rendered if `canCreate` is true (delete is gated by the same permission as create for now). Calls `removeDeploy(d.id)`. Hover: `text-error bg-error-light`.

UPLOADER STRIP

A horizontal rule (`border-t border-border/60`) separates the title block from a strip showing who uploaded:

- 7×7 `<Avatar>` with the uploader's initials.
- Two-line text block: bold full name above, muted role label below (omitted if absent).
- Right-aligned timestamp: relative time (e.g. *3h ago*) + literal date-time (e.g. *May 13, 8:42 AM*), separated by `.`. Both in `text-xs text-text-muted tabular-nums`.

CHANGELOG

Last block of the hero card. See §1.8 for the changelog component spec.

1.7 Timeline list

All filtered deploys after the first render in a vertical timeline below the hero.

THE RAIL

A single 1px vertical line behind every timeline dot, positioned with `insetInlineStart: 19` and stretched from `top-2 bottom-2`. RTL-aware via logical property.

EACH ROW

A flex row with two children:

- **Timeline dot** — 40×40 rounded-lg with the file-type icon centered. The dot's background and text color are derived from the deploy's status:
 - **Ready** → `bg-card text-text-dark border border-border`
 - **Deprecated** → `bg-warning-light text-warning`
 - **Rolled Back** → `bg-error-light text-error`

A 4px `ring-background` separates the dot from the rail behind it so the line "ends" at each dot rather than passing through.

- **Card** — `flex-1 hover:border-primary/30 transition-colors`, padding `p-4`. Layout:
 - Top row (flex-wrap): title (sm bold), version (mono primary), status badge, environment indicator dot+label, and right-aligned Download + Delete (smaller variants).
 - File meta line — same shape as the hero's, but smaller (`text-[11px]`).
 - Inline uploader: 5×5 avatar, name, separator, relative time.
 - Changelog (see §1.8).

1.8 Changelog component

Used identically inside the hero card and every timeline card. Renders as a flat list under the rest of the card content.

BULLET RENDERING

Each change is one `<li>`: a 1×1 primary-colored dot (margin-top tuned to align with the first line of text), then the text in `text-xs text-text-secondary leading-relaxed`.

INITIAL VISIBLE WINDOW

By default, only the first **2 bullets** are visible. If the deploy has 3 or more changes, a "Show N more" button appears below the list (where N = `changes.length - 2`).

SHOW / COLLAPSE TOGGLE

A small text button with a `ChevronDown` icon. Clicking it adds the deploy's id to a `Set<string>` stored in `expanded` state at the page level. The chevron rotates 180° via `rotate-180` conditional class when expanded. The button label switches between *"Show N more"* and *"Collapse"*.

EMPTY-CHANGES GUARD

If `changes.length === 0`, the entire block returns null. Submitting an upload with no bullets is normalized to `["No changelog provided"]` by the dialog, so this edge is rare.

1.9 Upload dialog

Component: `src/views/deploy/UploadDeployDialog.tsx`. Modal centered on the page, max-width `max-w-xl`. Mounted permanently in the page; visibility controlled by `uploadOpen` state.

1.9.1 DIALOG HEADER

Title: `UploadCloud` icon in primary + "New Deploy". Description: *"Upload a build artifact. Managers receive a notification when this is submitted."*

1.9.2 DRAG-AND-DROP ZONE

Top of the form. A `<div>` with dashed border (`border-2 border-dashed`) and padding `p-6`. State drives its visual:

STATE	VISUAL
Idle	<code>border-border</code> , transparent bg, hover changes to <code>border-primary/40 bg-muted/40</code>
Dragging over	<code>border-primary bg-primary/8</code>
File loaded	<code>border-primary/40 bg-primary/4</code> , content swaps to file-row representation (see below)

Click anywhere in the zone triggers `fileInputRef.current?.click()` to open the OS file picker. The hidden `<input type="file">` sits inside the zone.

Drag handlers

- `onDragOver` — `e.preventDefault()` + `setIsDragging(true)`
- `onDragLeave` — `e.preventDefault()` + `setIsDragging(false)`
- `onDrop` — `e.preventDefault()` + `setIsDragging(false)` + `acceptFile(e.dataTransfer.files?.[0])`

Idle-state content

Centered `UploadCloud` icon (color transitions to primary when dragging), then *"Drag a build file or click to browse"* (switches to *"Drop the file here"* while dragging), then a small *"Up to 250 MB · any format"* hint.

File-loaded representation

Switches the zone's children to a single horizontal row:

- 48×48 primary-filled rounded box with a generic `File` icon.
- Filename (truncated, semibold) and below it the size in MB (2 decimal places) plus the MIME type, in mono-tabular text.

- An X button (right-aligned). Clicking **stops propagation** so the zone's click handler doesn't re-open the file picker, and clears the file state.

1.9.3 TITLE INPUT

Required. Standard `<Input>`. Placeholder: `"API Service"`.

Auto-fill behavior: when a file is dropped or picked and the title is empty, `acceptFile` derives a default title from the filename:

```
const base = f.name
  .replace(/\.[^.]$/, "")      // strip extension
  .replace(/[-_]+/g, " ")      // dashes/underscores → spaces
  .trim();
setTitle(base.charAt(0).toUpperCase() + base.slice(1));
```

e.g. `api-service-2.14.0.tar.gz` → `"Api service 2.14.0"`.

1.9.4 VERSION INPUT

Optional. Free text. Placeholder: `v2.14.0`.

1.9.5 ENVIRONMENT SELECT

Three options — Production / Staging / Preview — backed by the `DeployEnvironment` enum. Default: `Production`.

1.9.6 CHANGES TEXTAREA

5-row mono-font textarea with placeholder:

```
One bullet per line:
Added GraphQL endpoint...
Fixed CVE-2026-1184...
```

Hint below: *"One line per change. Bullets are added automatically."*

Normalization on submit

```
const bullets = changes
  .split("\n")
  .map(s => s.replace(/^[\s-]*\s+/, "").trim())
  .filter(Boolean);
```

So users who paste a real bullet list (with `•`, `-`, or `*` prefixes) get clean text. Empty lines are dropped. If the result is empty, the deploy is created with `changes: ["No changelog provided"]`.

1.9.7 VALIDATION & SIZE CAP

CONDITION	BEHAVIOR
<code>!file</code>	<code>canSubmit = false</code> → submit button disabled
<code>!title.trim()</code>	<code>canSubmit = false</code> → submit button disabled
<code>file.size > 250 × 1024 × 1024</code>	<code>fileTooBig = true</code> → red inline error <i>"File exceeds the 250 MB demo limit."</i> ; submit disabled

1.9.8 SUBMIT BEHAVIOR

On click of "Submit Deploy":

1. Generates a new id: `"dep-" + Math.random().toString(36).slice(2, 10)`.
2. Constructs a `DeployInterface` using the form fields + the authenticated `user` as `uploaded_by`.
3. Captures a Blob URL via `URL.createObjectURL(file)`; stores it on `download_url` so the Download button can re-deliver the file later in the session.
4. Calls `addDeploy(newDeploy)` on the store; the new entry lands at index 0.
5. Calls `onOpenChange(false)` to close the modal.

1.9.9 RESET ON CLOSE

A `useEffect` watching `open` clears all form state (`file`, `title`, `version`, `changes`, `environment`, `isDragging`) whenever the dialog closes — so opening it again starts fresh.

1.10 Status & environment models

DEPLOYSTATUS ENUM (`SRC/ENUMS/DEPLOYS.TS`)

VALUE	BADGE	MEANING
<code>ready</code>	Ready	The canonical live build. Show prominently. Permit downloads.
<code>deprecated</code>	Deprecated	Superseded by a newer build. Still downloadable for audit / rollback evaluation.
<code>rolled_back</code>	Rolled Back	Withdrawn from production. Keep the record but flag visually.

DEPLOYENVIRONMENT ENUM

VALUE	DOT COLOR	LABEL TONE
<code>production</code>	production	<code>text-success</code> + <code>bg-success</code> dot
<code>staging</code>	staging	<code>text-warning</code> + <code>bg-warning</code> dot
<code>preview</code>	preview	<code>text-primary</code> + <code>bg-primary</code> dot

1.11 File-type iconography

`iconForFile(fileName)` returns the Lucide component to display next to a deploy's title, based on the file's extension (lowercased, post-final-dot).

EXTENSIONS	ICON	DOMAIN
<code>.apk</code> <code>.ipa</code> <code>.aab</code>	<code>Smartphone</code>	Mobile app builds (Android, iOS, Android App Bundle)
<code>.zip</code> <code>.tar</code> <code>.gz</code> <code>.rar</code> <code>.7z</code> <code>.tgz</code> <code>.bz2</code>	<code>Archive</code>	Compressed bundles
<code>.dmg</code> <code>.pkg</code> <code>.deb</code> <code>.rpm</code> <code>.exe</code> <code>.msi</code>	<code>Package</code>	OS-level installers
anything else	<code>Package</code>	Generic fallback

1.12 Helper functions

FORMATSIZE(BYTES)

```
const formatSize = (bytes: number): string => {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 ** 2) return `${(bytes / 1024).toFixed(1)} KB`;
  if (bytes < 1024 ** 3) return `${(bytes / 1024 ** 2).toFixed(2)} MB`;
  return `${(bytes / 1024 ** 3).toFixed(2)} GB`;
};
```

Used for the per-deploy file size and the Total Size stat.

RELATIVETIME(ISO)

```
const relativeTime = (iso: string): string => {
  const diffMs = Date.now() - new Date(iso).getTime();
  const mins = Math.floor(diffMs / 60_000);
  if (mins < 1) return "just now";
  if (mins < 60) return `${mins} min ago`;
  const hours = Math.floor(mins / 60);
  if (hours < 24) return `${hours}h ago`;
  const days = Math.floor(hours / 24);
  if (days < 30) return `${days}d ago`;
  const months = Math.floor(days / 30);
  return `${months}mo ago`;
};
```

Output buckets: "just now", "X min ago", "Xh ago", "Xd ago", "Xmo ago". No year bucket because deploys older than ~12 months are an edge nobody cares about.

STATUSBADGE(STATUS)

```
const statusBadge = (status: DeployStatus) => {
  if (status === DeployStatus.Ready) return <Badge variant="success" />
  if (status === DeployStatus.Deprecated) return <Badge variant="warning" />
  return <Badge variant="error" />;
};
```

1.13 Download flow

The `handleDownload(d)` function decides at runtime whether the deploy has a live Blob URL or needs a synthesized fallback:

```

const handleDownload = (d: DeployInterface) =>{
  if (d.download_url) {
    // In-session upload: re-deliver the real file
    const a = document.createElement("a");
    a.href = d.download_url;
    a.download = d.file_name;
    document.body.appendChild(a);
    a.click();
    document.body.removeChild(a);
    return;
  }
  // Seed entry or post-refresh entry: synthesize a placeholder
  const blob = new Blob(
    [`# ${d.title} ${d.version ?? ""}\n\nDemo download – backend integration pending.\n`
    { type: "text/plain" }
  );
  const url = URL.createObjectURL(blob);
  // ...same anchor-click pattern, with `.txt` extension
  URL.revokeObjectURL(url);
};

```

Net result: the Download button always delivers a file. Seeded entries hand you a stub `.txt`; in-session uploads hand you the original.

1.14 Empty states

CONDITION	UI
<code>deploys.length === 0</code>	<code><EmptyState icon={Rocket} title="No deploys yet" description="Upload your first build to start the changelog." /></code>
<code>filtered.length === 0</code> but <code>deploys.length > 0</code>	Same component, same copy. (TODO: a "Try clearing filters" variant could be added later.)
<code>filtered.length === 1</code>	Only the hero card renders; no timeline.

1.15 Data model

```
export interface DeployUploaderInterface {
  id: string;
  full_name: string;
  avatar_initials: string;
  role_label?: string | null;
}

export interface DeployInterface {
  id: string;
  title: string;
  version: string | null;
  file_name: string;
  file_size: number;           // bytes
  file_type: string;           // MIME or extension
  changes: string[];           // bullet array
  environment: DeployEnvironment;
  status: DeployStatus;
  uploaded_by: DeployUploaderInterface;
  uploaded_at: string;         // ISO timestamp
  /** Blob URL – present only for items created this session. */
  download_url?: string | null;
}

export interface UploadDeployPayloadInterface {
  title: string;
  version?: string | null;
  file: File;
  changes: string[];
  environment: DeployEnvironment;
}
```

1.16 Store

File: `src/store/deploy.ts`. Zustand store, in-memory only.

```
interface DeployStore {
  deploys: DeployInterface[];
  addDeploy: (deploy: DeployInterface) => void;
  removeDeploy: (id: string) => void;
}

export const useDeployStore = create<DeployStore>((set) => ({
  deploys: seedDeploys,
  addDeploy: (deploy) => set((s) => ({ deploys: [deploy, ...s.deploys] })),
  removeDeploy: (id) => set((s) => ({ deploys: s.deploys.filter((d) => d.id !== id) })),
}));
```

No update/patch action — current UI doesn't expose editing existing deploys. Add one when the backend defines a status-mutation endpoint.

1.17 Seed data inventory

Seven seeded deploys in `src/data/seed/deploys.ts`, spanning all environments and statuses. Time offsets are computed at import time via `hoursAgo(n)` / `daysAgo(n)` helpers so the seed always feels fresh.

#	TITLE	VERSION	ENV	STATUS	AGE	SIZE
1	API Service	v2.14.0	production	ready	3h ago	45.0 MB
2	Web Dashboard	v4.2.1	production	ready	7h ago	12.0 MB
3	Mobile App (Android)	v1.8.2-beta	staging	ready	1d ago	27.0 MB
4	ML Pipeline	v0.9.0	staging	ready	2d ago	149.0 MB
5	API Service	v2.13.0	production	deprecated	5d ago	44.5 MB
6	Web Dashboard	v4.2.0	production	rolled back	6d ago	11.7 MB
7	Mobile App (iOS)	v1.8.0	production	ready	11d ago	30.0 MB

Per-deploy changelogs are realistic and varied — see `src/data/seed/deploys.ts` for the full content. Examples:

- **API Service v2.14.0** — "Added GraphQL endpoint for sprint analytics", "Reduced cold-start by 38% via lazy module loading", "Patched CVE-2026-1184 in axios upstream", "Migrated background jobs to BullMQ"
- **Web Dashboard v4.2.0 (rolled back)** — "Reverted: bug in topbar caused project selector to lose state on refresh"

02 Reminders

Urgency-driven countdown cards for anything that expires. Multi-interval notifications, color-shifting design.

2.1 Purpose & user roles

The platform team had no place to track things that quietly expire and break: SSL certificates, Apple Developer renewals, AWS reservations, GitHub Enterprise seats, domain registrations. The Reminders module gives managers a place to capture each item with its expiry date and configurable notification intervals. The list itself becomes a dashboard of organizational urgency.

Primary user	Managers / leads creating and triaging reminders.
Secondary user	Any authenticated user — read-only.
Write access	Gated by <code>p.tasks.create()</code> as a stand-in until <code>p.reminders.create()</code> is added.
Page path	<code>/reminders</code>
Sidebar group	Secondary (below divider), with the BellRing icon.

2.2 Information architecture

1. Page header — title, description, primary action.
2. Statistics row — four count tiles.
3. Tabs + filter card — status tabs at top, search + urgency + sort below.
4. Reminder grid — 2-column on `md+`, single column below.
5. Add/Edit dialog — modal for create + edit (two instances mounted, one for each).

2.3 Page header

- **Title:** `"Reminders"`
- **Description:** *"Subscription, certificate, and renewal alerts. Get pinged at each interval before expiry."*
- **Primary action:** button with `BellPlus` icon and label `"New Reminder"`. Opens the Add dialog. Gated by permission.

2.4 Statistics row

Four tiles spanning the page width. Each derived from `annotated` (see §2.10 — reminders post-status-annotation).

TILE	SOURCE FORMULA	COLOR
Total	<code>annotated.length</code>	text-text-dark
Active	<code>annotated.filter(r => r._status === ReminderStatus.Active).length</code>	text-primary
This Week	<code>annotated.filter(r => r._days >= 0 && r._days <= 7 && r._status === Active).length</code>	text-warning
Critical (≤3d)	<code>annotated.filter(r => r._days >= 0 && r._days <= 3 && r._status === Active).length</code>	text-error

All counts in one `useMemo` keyed on `[annotated]`.

2.5 Status tabs

A `<Tabs>` control inside the filter card. Four triggers, each with the status name + a small tabular-numerals count.

TAB	PREDICATE
All	no filter
Active	<code>r._status === ReminderStatus.Active</code>
Expired	<code>r._status === ReminderStatus.Expired</code>
Dismissed	<code>r._status === ReminderStatus.Dismissed</code>

2.6 Filter bar

SEARCH INPUT

Same pattern as Deploys. Matches against `title` and `description` (the latter coerced to `""` if null), case-insensitive substring.

URGENCY SELECT

Six options that correspond to the `urgencyFor(days).tone` output:

- All Urgencies
- Due Today (`tone === "today"`)
- Critical (≤3d) (`tone === "critical"`)
- This Week (`tone === "warning"`)

- Soon ($\leq 30d$) (`tone === "soon"`)
- Comfortable (`tone === "calm"`)

SORT SELECT

Three options:

VALUE	COMPARATOR
<code>date-asc</code> (Soonest first — default)	<code>a._days - b._days</code>
<code>date-desc</code> (Latest first)	<code>b._days - a._days</code>
<code>created</code> (Recently added)	<code>new Date(b.created_at).getTime() - new Date(a.created_at).getTime()</code>

CLEAR BUTTON + COUNT INDICATOR

Same pattern as Deploys. Clear resets `search`, `urgencyFilter`, and `tab` (but leaves `sortBy` as the user's preference).

2.7 Reminder card anatomy

Each filtered reminder renders as a card with an unusual horizontal split: a countdown column on the inline-start side, a content column filling the rest.

CONTAINER

`<Card>` with `transition-all duration-200 overflow-hidden`, plus a urgency-derived hover-border class (e.g. `hover:border-error/40` for critical) and an optional drop-shadow glow class (only for today + critical tones). When the reminder is dismissed or expired, the entire card gets `opacity-60`.

COUNTDOWN COLUMN (INLINE-START)

`min-w-[88px]`, vertical-centered, with `bg-gradient-to-br from-transparent to-muted/30`. Inside:

- A 4px urgency-colored rail absolutely positioned along the inline-start edge.
- Three states for the number:
 - **Active and future** — large 3xl bold tabular-numeral days count, urgency-colored. Below: "DAYS LEFT" or "DAY LEFT" (singular when `days === 1`) in tiny uppercase tracking-wider.
 - **Today** — text reads "TODAY" instead of a number, in `text-2xl` bold error red. Below: "EXPIRES".
 - **Expired** — large 3xl bold tabular-numeral *absolute* value (i.e. days since expiry, not negative). Color: muted. Below: "DAYS AGO".

CONTENT COLUMN

Flex-1 with `p-4`. Top row: urgency badge + dismissed badge (conditional) on the left, actions menu (vertical dots) on the right. Body: title (sm bold), description (xs secondary, line-clamp-2). Bottom meta row (flex-wrap):

- Calendar icon + the formatted expire date.
- Bell icon + a row of small primary-tinted "interval pills" — each is a `min-w-[26px] h-5` rounded box with `bg-primary/10 text-primary` showing the formatted interval (`1w`, `3d`, etc.).
- Right-aligned: 5×5 avatar of the creator with full name (truncated to `max-w-[120px]`).

2.8 Urgency model

The entire visual identity of every card is driven by two helper functions.

DAYSUNTIL(ISO): NUMBER

```
const daysUntil = (iso: string): number => {
  const target = new Date(iso);
  target.setHours(23, 59, 59, 999); // end-of-day so today is day 0, not -1
  const now = new Date();
  return Math.ceil((target.getTime() - now.getTime()) / DAY_MS);
};
```

The `setHours` step makes "expires today" actually return 0, not -1 if the user opens the page at noon. `Math.ceil` means a reminder 1.4 days away rounds to 2, not 1 — biases toward "more time" so urgency is never overstated.

URGENCYFOR(DAYS): URGENCY

Returns a struct with `tone`, `label`, `color` (text class), `ring` (bg class for the rail), `border` (hover border), `progress` (color for progress bars), and `glow` (drop-shadow class, only set for high-urgency states).

-4 EXPIRED	TODAY EXPIRES	3 CRITICAL	7 THIS WEEK	14 SOON	45 COMFORTABLE
DAYS	TONE	COLOR	RING	BORDER ON HOVER	GLOW
< 0	expired	text-text-faint	bg-text-muted	hover:border-text-muted	—
= 0	today	text-error	bg-error	hover:border-error/40	0 4px 20px -8px rgba(239,68,68,0.4)
1-3	critical	text-error	bg-error	hover:border-error/40	0 4px 20px -8px rgba(239,68,68,0.3)
4-7	warning	text-warning	bg-warning	hover:border-warning/40	—
8-30	soon	text-primary	bg-primary	hover:border-primary/40	—
> 30	calm	text-success	bg-success	hover:border-success/40	—

Why these ranges:

- **3-day threshold** aligns with the `NOTIFY_PRESETS` shortest preset — anything inside 3 days is "you can't unbreak this if you forget".
- **7-day threshold** matches the most common "1 week" notification preset.
- **30-day threshold** matches the longest preset; beyond that, the team has comfortable lead time.

2.9 Add / Edit dialog

File: `src/views/reminders/AddReminderDialog.tsx`. **One component, two roles** — the page mounts it twice, once for create and once for edit. Each mount distinguishes by whether the `initial` prop is set.

2.9.1 DIALOG HEADER

Title: `BellPlus` icon in primary + "New Reminder" or "Edit Reminder". Description: *"We'll notify you at each selected interval before the expiry date."*

2.9.2 TITLE INPUT

Required. Placeholder: `"SSL certificate renewal"`. No special validation beyond non-empty.

2.9.3 DESCRIPTION TEXTAREA

Optional. 3 rows. Placeholder: *"Context, links, or any details you want surfaced when this fires."*

2.9.4 EXPIRE DATE INPUT

Required. `type="date"`. `min={todayISO}` so the browser native picker disables past dates (this is enforcement-by-UI; the form will still accept a programmatically-set past date but the picker can't produce one).

2.9.5 NOTIFY-BEFORE CHIP PICKER

The heart of the dialog. Above the chips: the label *"Notify me before"* with a red asterisk. Below: a flex-wrap row of pill chips.

Presets

The `NOTIFY_PRESETS` enum is a tuple `[1, 2, 3, 7, 14, 30]` as `const`. Each value renders as a chip whose label is the output of `labelFor(days)`:

```
const labelFor = (days: number): string => {
  if (days === 1) return "1d";
  if (days < 7) return `${days}d`;           // 2d, 3d
  if (days === 7) return "1w";
  if (days === 14) return "2w";
  if (days % 7 === 0 && days < 30) return `${days / 7}w`;
  if (days === 30) return "1m";
  return `${days}d`;                         // custom days
};
```

Chip visual states:

STATE	STYLE
Inactive	<code>bg-card text-text-dark border-border</code> , hover: <code>border-primary/40 bg-primary/5</code>
Active (selected)	<code>bg-primary text-primary-foreground border-primary shadow-sm</code>
Custom (not in presets)	Active style + a small <code>x</code> button to remove it

Custom days input

Below the chips, a row containing:

- A `type="number"` input with `min=1 max=365`, width `w-32`, placeholder `"Custom days..."`.
- An **Add** button (small, outlined). Disabled while the input is empty.
- Right-aligned: *"N intervals"* (singular "interval" when count is 1).

Pressing `Enter` in the number input is the same as clicking Add (with `preventDefault` to stop form submission).

Add custom interval logic

```
const addCustom = () => {
  const n = parseInt(customInput.trim(), 10);
  if (!isNaN(n) && n > 0 && !intervals.includes(n)) {
    setIntervals([...intervals, n].sort((a, b) => b - a));
    setCustomInput("");
  }
};
```

Guards: NaN ignored, ≤ 0 ignored, duplicates ignored. Result is always sorted descending so the chips render largest-first in the UI.

2.9.6 VALIDATION

FIELD	REQUIRED	EMPTY-STATE BEHAVIOR
Title	Yes	Submit disabled
Description	No	Submitted as <code>undefined</code>
Expires on	Yes	Submit disabled
Intervals	≥ 1	Submit disabled (the system needs at least one notification to fire)

2.9.7 SUBMIT BEHAVIOR

1. If `initial` is set: calls `updateReminder(initial.id, payload)`.
2. Otherwise: calls `addReminder(payload, by)` where `by` is the authenticated user. The store generates the id, sets `status: Active`, sets `created_at` to now, and sorts `notify_before_days` descending.
3. Closes the dialog.

2.9.8 RESET ON OPEN

A `useEffect` on `[open, initial]` repopulates every field whenever the dialog opens:

```
setTitle(initial?.title ?? "");
setDescription(initial?.description ?? "");
setExpiresAt(initial?.expires_at ?? "");
setIntervals(initial?.notify_before_days ?? [7, 1]); // sensible default for create
setCustomInput("");
```


2.10 Auto-expiration logic

Reminders past their expiry should appear as `Expired` even if the stored status is still `Active`. This is an **annotation**, not a persisted change — important for UX rationale below.

ANNOTATION PIPELINE

```
const annotated = useMemo(() => {
  return reminders.map((r) => {
    const days = daysUntil(r.expires_at);
    const isExpired = days < 0;
    const effectiveStatus =
      isExpired && r.status === ReminderStatus.Active
        ? ReminderStatus.Expired
        : r.status;
    return { ...r, _days: days, _status: effectiveStatus };
  });
}, [reminders]);
```

Why not just mutate the store? Because the user might extend the expiry date later — bumping `expires_at` back into the future. If we'd flipped the stored status to `Expired`, the reminder would stay flagged expired until manually reactivated. By keeping the stored status as `Active` and computing `_status` on render, the system "heals" automatically when the date is changed.

The annotated array (carrying `_days` and `_status`) flows through every filter, sort, and rendering decision. The raw `reminders` array is never read directly by the view.

2.11 Interval format display

The page-level helper is identical to the dialog's `labelFor` — there are *two copies* intentionally so each file is self-contained.

```
const formatInterval = (d: number): string => {
  if (d === 1) return "1d";
  if (d < 7) return `${d}d`;
  if (d === 7) return "1w";
  if (d === 14) return "2w";
  if (d % 7 === 0 && d < 30) return `${d / 7}w`;
  if (d === 30) return "1m";
  return `${d}d`;
};
```

Examples:

DAYS	LABEL	DAYS	LABEL
1	1d	14	2w
2	2d	21	3w
3	3d	30	1m
7	1w	60	60d

2.12 Card actions

Top-right of each card: a `MoreHorizontal` dropdown menu (only when `canEdit` is true).

ACTION	ICON	CONDITION	EFFECT
Edit	Pencil	always	Opens the dialog with this reminder as <code>initial</code>
Dismiss	BellOff	not currently dismissed	<code>setStatus(id, Dismissed)</code> — card dims to opacity 60%
Reactivate	Bell	currently dismissed	<code>setStatus(id, Active)</code> — card returns to normal opacity
Delete	Trash2	always	<code>removeReminder(id)</code> — removed from store

The Delete item has `text-error focus:text-error` classes so it reads as destructive in the menu.

2.13 Analytics & counts

All counts are derived from the annotated array inside one `useMemo`:

```
const counts = useMemo(() => ({
  all:      annotated.length,
  active:   annotated.filter((r) => r._status === ReminderStatus.Active).length,
  expired:  annotated.filter((r) => r._status === ReminderStatus.Expired).length,
  dismissed: annotated.filter((r) => r._status === ReminderStatus.Dismissed).length,
  critical: annotated.filter((r) => r._days >= 0 && r._days <= 3 && r._status === Active).length,
  thisWeek: annotated.filter((r) => r._days >= 0 && r._days <= 7 && r._status === Active).length,
}), [annotated]);
```

These feed both the four stat tiles and the badge counts on the four status tabs.

2.14 Empty states

CONDITION	UI
No reminders at all	<pre><EmptyState icon={Bell} title="No reminders match" description="Create your first reminder to get notified before things expire." /></pre>
Filters active, zero matches	Same component, description: <i>"Try clearing filters."</i>

2.15 Data model

```
export interface ReminderCreatorInterface {
  id: string;
  full_name: string;
  avatar_initials: string;
}

export interface ReminderInterface {
  id: string;
  title: string;
  description: string | null;
  expires_at: string; // ISO yyyy-mm-dd or full ISO
  /** Days before expiration to fire a notification. Multiple allowed.
   * Stored sorted descending so the largest interval is always first. */
  notify_before_days: number[];
  status: ReminderStatus; // active | expired | dismissed
  created_by: ReminderCreatorInterface;
  created_at: string;
}

export interface CreateReminderPayloadInterface {
  title: string;
  description?: string;
  expires_at: string;
  notify_before_days: number[];
}

export interface UpdateReminderPayloadInterface extends Partial<CreateReminderPayloadInterface> {
  status?: ReminderStatus;
}
```

2.16 Store

File: `src/store/reminders.ts`. Zustand store with full CRUD.

```
interface RemindersStore {
  reminders: ReminderInterface[];
  addReminder: (payload, by) => ReminderInterface;
  updateReminder: (id, patch) => void;
  removeReminder: (id) => void;
  setStatus: (id, status) => void;
}
```

ADDREMINDER

Generates an id, sets `status: Active`, sets `created_at: new Date().toISOString()`, sorts `notify_before_days` descending, prepends to the array, returns the new entry. Also coerces `description ?? null` for consistency with the seeded shape.

UPDATEREMINDER

Patches the matching entry. Handles two special fields:

- If `notify_before_days` is in the patch, it's re-sorted descending.
- If `description` is in the patch, it's coerced from `undefined` to `null`.

SETSTATUS

Convenience for the Dismiss / Reactivate flow — just sets the status without touching anything else.

2.17 Seed data inventory

Six seeded reminders in `src/data/seed/reminders.ts`. The `inDays(n)` helper computes `expires_at` dates relative to "today" at import time, so the urgency colors look right whenever the seed loads.

#	TITLE	DAYS UNTIL	INTERVALS	STATUS
1	SSL Certificate — api.twindix.com	3 (critical)	7, 2, 1	Active
2	Apple Developer Program renewal	14 (warning)	30, 14, 7, 1	Active
3	AWS reserved instance term	45 (soon)	30, 14, 7	Active
4	GitHub Enterprise subscription	78 (soon)	30, 14, 7, 2	Active
5	Domain registration — twindix.com	412 (comfortable)	30, 14	Active
6	Figma Org plan renewal	-4 (expired)	14, 7, 1	Expired

The descriptions are deliberately practical so screenshots / demos read as plausible — e.g. *"1-year RI on the API cluster. Decide: renew, switch to Savings Plan, or downsize."*

Shared architecture

Conventions used by both modules.

3.1 Layer responsibilities

LAYER	PATH	RESPONSIBILITY
Enum	<code>src/enums/<module>.ts</code>	Categorical values: status, environment, presets.
Interface	<code>src/interfaces/<module>/index.ts</code>	TS shapes for the entity and its payloads.
Seed	<code>src/data/seed/<module>.ts</code>	Initial in-memory list. Goes away once backend ships.
Store	<code>src/store/<module>.ts</code>	Zustand store with CRUD. Seed-initialized.
View	<code>src/views/<module>/index.tsx</code>	The page component.
Dialog	<code>src/views/<module>/<Action>Dialog.tsx</code>	Modal forms.
Route	<code>src/data/routes.ts</code> + <code>src/routes/router.tsx</code>	Path registration.
Sidebar	<code>src/data/sidebar.ts</code>	Navigation entry with icon.

3.2 Routing

ROUTE KEY	PATH	ELEMENT
<code>routesData.deploys</code>	<code>/deploys</code>	<code><DeploysView /></code>
<code>routesData.reminders</code>	<code>/reminders</code>	<code><RemindersView /></code>

Both routes are added inside the protected `<DashboardLayout>` subtree — guarded by `<ProtectedRoute>` and rendered with the sidebar/topbar shell.

3.3 Sidebar entries

Both go into `sidebarNewItems` (the secondary group, rendered below a divider in the sidebar):

```
{ label: "Deploys", path: routesData.deploys, icon: Rocket },  
{ label: "Reminders", path: routesData.reminders, icon: BellRing },
```

Icon choices were specifically:

- **Rocket** — "deploy = launch" is the most universally-understood metaphor. Distinct from the cargo/box icons used elsewhere.
- **BellRing** — visually distinct from `Bell` (already used by Alerts) but in the same icon family, so users intuitively associate it with notifications.

3.4 Theming & visual conventions

- Both modules use only the existing Tailwind theme tokens — `primary`, `success`, `warning`, `error`, `text-dark`, `text-muted`, `text-faint`, `card`, `border`, `muted`. No module-specific design tokens defined.
- Tabular numerals (`font-variant-numeric: tabular-nums`) on every numeric display — counters, sizes, days-until — so digits don't jitter as values change.
- Mono font for file names, version strings, and intervals.
- 10pt uppercase 0.12em-tracked labels for tiny meta lines, consistent with the editorial direction set elsewhere in the dashboard.

3.5 Animations & transitions

All animations are CSS-only and GPU-composited:

- Drop-zone state changes — `transition-all duration-200`
- Card hovers — `transition-colors` (for border tint) and `transition-all duration-200` (when card translates)
- Changelog expand — height not animated (preserves layout stability); only the chevron rotates via `transition-transform`
- Dialog open/close — managed by the underlying Radix Dialog primitive; default fade + scale

3.6 Accessibility

- Every icon-only button has an `aria-label` (Download, Delete, Remove file, etc.).
- Status tabs and select widgets use the existing Radix primitives — full keyboard nav and screen-reader semantics inherited.

- The drop-zone fallback is always a real `<input type="file">`; users without drag-and-drop support (keyboard / assistive tech) can still upload.
- Color is never the only signal: every urgency state in Reminders also has a text label (TODAY, CRITICAL, THIS WEEK, ...) and a distinct numeric.

3.7 Permissions

Both modules currently route create/delete gating through `p.tasks.create()` — a deliberate stand-in. The proper fix when backend ships:

1. Add policy predicates: `p.deploys.create()`, `p.deploys.delete()`, `p.reminders.create()`, `p.reminders.edit()`, `p.reminders.delete()`.
2. Update the two views to use the new predicates.
3. Per the project's permission strategy: components *hide* rather than *disable* for missing permissions (with the existing exception of viewer-on-own-profile).

Backend integration roadmap

What needs to ship server-side for each module to graduate from "UI complete" to "production".

4.1 Deploys endpoints (proposed)

METHOD	ENDPOINT	PURPOSE
GET	/api/deploys	List, paginated, filterable by <code>environment</code> , <code>status</code> , <code>uploaded_by</code> , <code>search</code> .
POST	/api/deploys	Multipart upload — file + JSON metadata (<code>title</code> , <code>version</code> , <code>environment</code> , <code>changes[]</code>).
GET	/api/deploys/{id}	Detail.
GET	/api/deploys/{id}/download	Stream the binary, or 302 to a signed S3/R2/GCS URL.
PATCH	/api/deploys/{id}/status	Change status (mark <i>deprecated</i> / <i>rolled_back</i>). Optional.
DELETE	/api/deploys/{id}	Permanent delete. Admin only.

Side effects

- **Notify managers** on every successful `POST`. Likely an internal event the existing Alerts/Notifications pipeline already handles — needs the deploy-uploaded event type added.
- **Object storage** — files go to S3/R2/GCS with a signed-URL download flow. The frontend's `download_url` field becomes the signed URL returned by the list endpoint.
- **Antivirus / scanning** — uploads should be scanned before being downloadable. Hide cards with status `scanning` or `quarantined` from non-admins.
- **Size enforcement** — the 250 MB UI cap is advisory; the backend should enforce real limits per environment.

4.2 Reminders endpoints (proposed)

METHOD	ENDPOINT	PURPOSE
GET	/api/reminders	List, paginated, filterable by status, urgency, search.
POST	/api/reminders	Create. Body: { title, description, expires_at, notify_before_days[] }.
GET	/api/reminders/{id}	Detail.
PUT	/api/reminders/{id}	Full update.
PATCH	/api/reminders/{id}/dismiss	Set status to dismissed.
PATCH	/api/reminders/{id}/reactivate	Set status to active.
DELETE	/api/reminders/{id}	Delete.

Scheduler

- A background job (cron, BullMQ, Sidekiq) runs daily and finds every reminder where `expires_at - today` equals any value in `notify_before_days`. Each match fires a notification.
- The same job promotes `active` reminders past their expiry to `expired` server-side, so the frontend's auto-annotate can be removed.
- Notifications go to the reminder's `created_by` user. Optionally extendable to team-wide alerts.

4.3 Migration sequence

Both modules follow the established pattern used by auth, users, projects, sprints, blockers, decisions, etc.

1. Add endpoint paths to `src/data/apis.ts` under `deploys` and `reminders` keys.
2. Create `src/services/deploys/index.ts` and `src/services/reminders/index.ts` with one `handler` per endpoint, returning typed responses.
3. Add domain hooks: `useDeploysList`, `useCreateDeploy`, `useDeleteDeploy`, `useRemindersList`, `useCreateReminder`, `useUpdateReminder`, `useRemoveReminder`, `useSetReminderStatus`. Use `usePaginatedQuery` for the list hooks and `useMutationAction` for the rest — both already established primitives.
4. Swap the views from `useDeployStore` / `useRemindersStore` to the new hooks. The component code barely changes — mostly the destructure at the top of each view.
5. Remove the seed imports and the Zustand stores. Optionally keep the stores as a local-cache layer if optimistic mutations are desired.
6. Add proper policy predicates (`p.deploys.*`, `p.reminders.*`) and swap the `p.tasks.create()` stand-ins.

7. Wire the deploy-uploaded backend event to the existing notification system.

Appendices

5.1 File index — added by these modules

Deploys (8 files)

src/	
enums/	
+ deploys.ts	DeployStatus, DeployEnvironment
interfaces/deploys/	
+ index.ts	DeployInterface, payloads
data/seed/	
+ deploys.ts	7 seeded deploys
store/	
+ deploy.ts	useDeployStore
views/deploys/	
+ index.tsx	Page: stats, filters, hero, timeline
+ UploadDeployDialog.tsx	Drag-drop upload form

Reminders (6 files)

src/	
enums/	
+ reminders.ts	ReminderStatus, NOTIFY_PRESETS
interfaces/reminders/	
+ index.ts	ReminderInterface, payloads
data/seed/	
+ reminders.ts	6 seeded reminders
store/	
+ reminders.ts	useRemindersStore (full CRUD)
views/reminders/	
+ index.tsx	Page: stats, tabs, urgency cards
+ AddReminderDialog.tsx	Multi-interval picker + custom days

Shared barrels modified (8 files)

src/	
enums/index.ts	+ DeployStatus, DeployEnvironment, + ReminderStatus, NOTIFY_PRESETS
interfaces/index.ts	+ re-export deploys + reminders types
data/seed/index.ts	+ seedDeploys, seedReminders
store/index.ts	+ useDeployStore, useRemindersStore
data/routes.ts	+ deploys: "/deploys" + reminders: "/reminders"
data/sidebar.ts	+ Rocket, BellRing icons + entries
routes/router.tsx	+ DeploysView, RemindersView routes
views/index.ts	+ re-export the new views

5.2 Commit history

SHA	SUBJECT
efe03c6	feat: add deploys module — upload, timeline, latest hero
3b24284	feat: add reminders module — countdown urgency cards

5.3 Glossary

TERM	MEANING
Annotation	A derived field (e.g. <code>_status</code> , <code>_days</code>) computed on render and never persisted to the store. Used by reminders to mark items as <code>Expired</code> without losing the stored <code>Active</code> state.
Blob URL	<code>URL.createObjectURL(file)</code> — a session-scoped URL that lets the browser re-deliver an in-memory <code>File</code> . Lives until <code>URL.revokeObjectURL</code> or page unload.
Critical (≤3d)	Reminders 0–3 days from expiry. Rendered with error-red color + drop-shadow glow.
Hero card	The first item of the filtered Deploys list, rendered with a larger, gradient-backed layout. Distinct from the timeline below.
Interval pill	The small primary-tinted chip showing a notify-before duration on a reminder card (<code>1w</code> , <code>3d</code> , etc.).
Lite endpoint	(Convention from the rest of the platform.) A version of a list endpoint that returns minimal fields for dropdown population. Not yet defined for these modules.
Tabular nums	CSS <code>font-variant-numeric: tabular-nums</code> — fixes digit width so numeric displays don't reflow when values change.
Tone	The 6-value urgency classification returned by <code>urgencyFor(days)</code> : expired, today, critical, warning, soon, calm. Drives card color + glow + label.
Urgency rail	The 4px vertical color bar on the inline-start edge of a reminder's countdown column. Color is derived from the urgency tone.